

## EXPERIMENT – 5

**Experiment Title:** Analysis of Addressing Modes through C Programs and GDB Debugger

**Aim:** To examine and understand the various addressing modes used in instruction execution by running C programs and observing their runtime behavior through the GDB debugger.

**Software Used:** C Programming Language, GCC Compiler (gcc), GDB Debugger, and Linux Terminal / VS Code

**Theory:**

### Addressing Modes

Addressing modes determine the method by which an instruction locates and retrieves its operands during program execution.

#### 1. Immediate Addressing Mode

In this mode, the operand is embedded directly within the instruction itself, eliminating the need for any additional memory access to retrieve it.

Example:

**result = a + 5;**

Explanation:

- The constant 5 is encoded directly into the instruction
- Execution is faster since no memory fetch for the operand is necessary

#### 2. Direct Addressing Mode

Here, the instruction holds the memory address of the operand explicitly. The processor goes directly to that address to read or write the value.

Example:

**result = a + b;**

Explanation:

- The processor accesses the memory locations holding a and b directly
- A memory access is required to retrieve the operand values

#### 3. Indirect Addressing Mode

In this mode, the instruction points to a memory location that itself holds the address of the actual operand. Pointers in C are the classic implementation of this concept.

Example:

```
ptr = &a;  
result = *ptr;
```

Explanation:

- ptr holds the memory address of variable a
- Dereferencing \*ptr retrieves the value stored at that address
- Offers greater flexibility at the cost of an additional level of memory access

#### 4. Indexed Addressing Mode

The effective address of the operand is computed by adding an index offset to a base address. Array traversal in C is the most common use case for this mode.

Example:

```
result = arr[1];
```

Explanation:

- The final address is derived from the array's base address plus the index offset
- Particularly efficient for accessing elements in contiguous memory structures

#### 5. Register Addressing Mode

Operands are held directly in CPU registers rather than in main memory, making this one of the fastest addressing modes available.

Example (Conceptual):

```
result = a + b;
```

Explanation:

- The compiler may allocate a and b to processor registers during execution
- Avoids the latency associated with memory reads and writes

#### 6. Register Indirect Addressing Mode

Rather than holding the operand itself, a register stores the memory address where the operand resides. The CPU then uses that address to fetch the actual value.

Example (Conceptual):

```
ptr = &a;  
result = *ptr;
```

## Role of GDB

GDB assists in understanding addressing modes by enabling the following:

- Step-by-step program execution for fine-grained observation
- Live inspection of variable values and memory contents
- Analysis of how individual instructions are executed at runtime
- Viewing the underlying assembly code through the disassemble command

## Program Code:

### Part A

```
#include <stdio.h>

int main(){
    int a=10,b=20;
    int result;
    int arr[3]={1,2,3};
    int *ptr;

    result=a+5;        // Immediate addressing
    result=a+b;        // Direct addressing
    ptr=&a;             // Indirect addressing
    result=*ptr;
    result=arr[1];     // Indexed addressing

    printf("Result : %d",result);
    return 0;
}
```

### Part B

```
#include <stdio.h>

int x=10;

int main(){
    int a=10,b=20;
    int result;
    int arr[3]={1,2,3};
    int *ptr;

    result=a+5;        // Immediate
    result=a+b;        // Direct
    result=x;          // Global variable (data segment)
    ptr=&a;             // Indirect
    result=*ptr;
    result=arr[1];     // Indexed

    printf("Result : %d",result);
    return 0;
}
```

## Procedure:

- Write the C program (lab1.c)
- Compile with debug flag:

**gcc -g lab1.c -o out1**

- Run program:

**./out1**

- Start GDB:

**gdb ./out1**

- Execute commands:
  - break main → Set breakpoint
  - run → Start execution
  - next → Step execution
  - print a → Check variable
  - print &a → Address of variable
  - print ptr → Pointer value
  - info registers → CPU registers
  - disassemble main → View assembly

## Output & Debugging:

### Part A

```
nitdelhi@w2:~/coalab$ vi lab1.c
nitdelhi@w2:~/coalab$ gcc -g lab1.c -o out1
nitdelhi@w2:~/coalab$ ./out
bash: ./out: No such file or directory
nitdelhi@w2:~/coalab$ ./out1
Result :2nitdelhi@w2:~/coalab$ gcc -g lab1.c -o out1
nitdelhi@w2:~/coalab$ gdb ./out1
GNU gdb (Ubuntu 9.2-0ubuntu1~20.04.2) 9.2
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<http://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./out1...
(gdb) break main
Breakpoint 1 at 0x1169: file lab1.c, line 2.
(gdb) run
Starting program: /home/nitdelhi/coalab/out1
```

```
Breakpoint 1, main () at lab1.c:2
2      int main(){
(gdb) run
The program being debugged has been started already.
Start it from the beginning? (y or n) y
Starting program: /home/nitdelhi/coalab/out1
```

```
Breakpoint 1, main () at lab1.c:2
2      int main(){
(gdb) next
3          int a=10,b=20;
(gdb) next
5          int arr[3]={1,2,3};
(gdb) next
7          result=a+5;
(gdb) print a
$1 = 10
(gdb) print &a
$2 = (int *) 0x7fffffffdd54
```

```
(gdb) print ptr
$3 = (int *) 0x0
(gdb) print *ptr
Cannot access memory at address 0x0
(gdb) print arr[1]
$4 = 2
(gdb) next
8          result=a+b;
(gdb) next
9          ptr=&a;
(gdb)
10         result=*ptr;
(gdb) print ptr
$5 = (int *) 0x7fffffffdd54
(gdb) print *ptr
$6 = 10
(gdb) next
11         result=arr[1];
(gdb) next
12         printf("Result :%d",result);
(gdb) next
13         return 0;
(gdb) next
15     }
(gdb) info registers
rax                0x0                0
rbx                0x555555555210        93824992236048
rcx                0x0                0
rdx                0x0                0
rsi                0x555555555600e        93824992239630
rdi                0x7ffff7fb07e0        140737353811936
rbp                0x7fffffffdd80        0x7fffffffdd80
rsp                0x7fffffffdd50        0x7fffffffdd50
r8                 0x0                0
r9                 0x9                9
r10                0x555555555600e        93824992239630
r11                0x7fffffffddc27        140737488346151
r12                0x555555555080        93824992235648
r13                0x7fffffffde70        140737488346736
r14                0x0                0
r15                0x0                0
rip                0x5555555551ed        0x5555555551ed <main+132>
eflags             0x206            [ PF IF ]
cs                 0x33            51
ss                 0x2b            43
ds                 0x0                0
es                 0x0                0
fs                 0x0                0
gs                 0x0                0
(gdb)
```

## Part B

```
(gdb) disassemble main
Dump of assembler code for function main:
0x000055555555169 <+0>:      endbr64
0x00005555555516d <+4>:      push   %rbp
0x00005555555516e <+5>:      mov    %rsp,%rbp
0x000055555555171 <+8>:      sub    $0x30,%rsp
0x000055555555175 <+12>:     mov    %fs:0x28,%rax
0x00005555555517e <+21>:     mov    %rax,-0x8(%rbp)
0x000055555555182 <+25>:     xor    %eax,%eax
0x000055555555184 <+27>:     movl   $0xa,-0x2c(%rbp)
0x00005555555518b <+34>:     movl   $0x14,-0x28(%rbp)
0x000055555555192 <+41>:     movl   $0x1,-0x14(%rbp)
0x000055555555199 <+48>:     movl   $0x2,-0x10(%rbp)
0x0000555555551a0 <+55>:     movl   $0x3,-0xc(%rbp)
0x0000555555551a7 <+62>:     mov    -0x2c(%rbp),%eax
0x0000555555551aa <+65>:     add    $0x5,%eax
0x0000555555551ad <+68>:     mov    %eax,-0x24(%rbp)
0x0000555555551b0 <+71>:     mov    -0x2c(%rbp),%edx
0x0000555555551b3 <+74>:     mov    -0x28(%rbp),%eax
0x0000555555551b6 <+77>:     add    %edx,%eax
0x0000555555551b8 <+79>:     mov    %eax,-0x24(%rbp)
0x0000555555551bb <+82>:     mov    0x2e4f(%rip),%eax      # 0x555555558010 <x>
0x0000555555551c1 <+88>:     mov    %eax,-0x24(%rbp)
0x0000555555551c4 <+91>:     lea    -0x2c(%rbp),%rax
0x0000555555551c8 <+95>:     mov    %rax,-0x20(%rbp)
0x0000555555551cc <+99>:     mov    -0x20(%rbp),%rax
0x0000555555551d0 <+103>:    mov    (%rax),%eax
0x0000555555551d2 <+105>:    mov    %eax,-0x24(%rbp)
0x0000555555551d5 <+108>:    mov    -0x10(%rbp),%eax
0x0000555555551d8 <+111>:    mov    %eax,-0x24(%rbp)
0x0000555555551db <+114>:    mov    -0x24(%rbp),%eax
0x0000555555551de <+117>:    mov    %eax,%esi
0x0000555555551e0 <+119>:    lea    0xe1d(%rip),%rdi      # 0x555555556004
0x0000555555551e7 <+126>:    mov    $0x0,%eax
0x0000555555551ec <+131>:    callq 0x55555555070 <printf@plt>
=> 0x0000555555551f1 <+136>:    mov    $0x0,%eax
0x0000555555551f6 <+141>:    mov    -0x8(%rbp),%rcx
0x0000555555551fa <+145>:    xor    %fs:0x28,%rcx
0x000055555555203 <+154>:    je     0x5555555520a <main+161>
0x000055555555205 <+156>:    callq 0x55555555060 <__stack_chk_fail@plt>
0x00005555555520a <+161>:    leaveq
0x00005555555520b <+162>:    retq
End of assembler dump.
```

### Observations:

- Immediate values are directly embedded in instructions
- Direct addressing accesses variables stored in registers/memory
- Indirect addressing uses pointer to access memory
- Indexed addressing uses array indexing
- Global variables are stored in data segment
- GDB shows low-level execution through registers and assembly

### Result:

- Successfully demonstrated different addressing modes
- Verified execution using GDB step-by-step debugging
- Observed how high-level C code maps to low-level instructions

**Conclusion:**

This experiment provided a clear understanding of various addressing modes and their implementation in C programs. Using GDB debugger, the execution of instructions was analyzed at a low level, helping to understand how different addressing techniques are used by the processor during program execution.

**Author:** Aayush Anand